Mangrove®

BOTS & BYTES · FRIDAY, APRIL 24, 2026

Build Your Own Trading Bot.

With Claude Code and the Mangrove API.

NASHVILLE · SPONSORED BY STAND WITH CRYPTO

Tim Darrah.

PhD · Vanderbilt professor · Mangrove co-founder.

I build AI trading bots and teach other people how. This workshop is the first installment of Bots & Bytes — a curriculum on driving Claude Code well, and using it to ship things that touch real systems (and sometimes real money) without blowing yourself up.

BY THE END OF TODAY

What you'll leave with.

01

A running paper-trading bot

Authored a strategy, backtested it, promoted to paper, watched it tick. Local on your laptop. Self-custody.

02

Fluency driving Claude Code

Models, plan mode, CLAUDE.md, tool calls, skills, hooks, MCP. The actual working vocabulary.

03

A mental model for AI safety in finance

Why the same agent that ships your app can drain your wallet. What to gate. What never to automate.

04

Access to the full curriculum

Chapters 6–8: real wallets, live trading on Base, monitoring and extending the bot. Today is the trailer.

WHILE I TALK · START HERE

Install now.

01
VS Code

→ code.visualstudio.com

Our editor for today. Download + install if you don't have it.

02
Claude Desktop

→ claude.com/download

Includes the Claude Code CLI — you'll likely already have it.

03
Mangrove API key

→ mangrovedeveloper.ai

Sign up, copy key. We'll paste it in step 4.

04
Clone + setup

```
git clone .../app-in-a-box.git defi-agent
cd defi-agent &&
./scripts/setup.sh
```

Requires Python 3.11+. Paste API key when prompted.

Stuck? Flag a neighbor — buddy system. Stuck worse? Flag Tim.

START HERE

What is Claude?

Claude is a family of AI models built by **Anthropic**. Like GPT from OpenAI or Gemini from Google — but with a different design posture: safety-first, aligned, careful about autonomy.

Three model sizes exist at any time — **Haiku**, **Sonnet**, **Opus** — a speed/capability tradeoff. You pick which one each task is worth.

Good at reasoning, writing, coding, and — the thing we care about today — driving multi-step agentic work.

THE TIER LADDER

HAIKU
fastest / cheapest

SONNET
balanced

OPUS
most capable / slowest

FOUR WAYS IN

Where you use Claude.

claude.ai

Web chat

The chat interface most people know. Type a question, get an answer. Good for writing, research, one-off coding questions.

CLAUDE DESKTOP

Desktop app

Same model as claude.ai, but lives on your machine. Can open files, browse local folders. What you installed in step 2. Ships with Claude Code.

CLAUDE CODE

Terminal for devs

Drop into any repo, type to Claude, watch it read/edit/commit. Agentic coding. THIS is what we're using today. The rest of the workshop lives here.

THE API

Programmatic

Build your own apps on top of Claude. What powers every product above — plus your own. Not covered today, but worth knowing it's there.

ZOOMING IN ON ONE ENTRY POINT

What Claude Code is.

A terminal-based, agentic coding assistant. You type natural language. Claude **reads your files, runs your commands, edits your code** — inside your actual repo, with your actual tools.

LOCAL

Runs on your laptop

Your code never leaves your machine. Claude Code talks to Anthropic's API for the model, but tool calls and file edits happen locally. Self-hosted where it matters.

VISIBLE

Nothing is hidden

Every tool call, every file edit, every result — shown inline as it happens. You read them. You approve them. No black-box behavior.

CUSTOMIZABLE

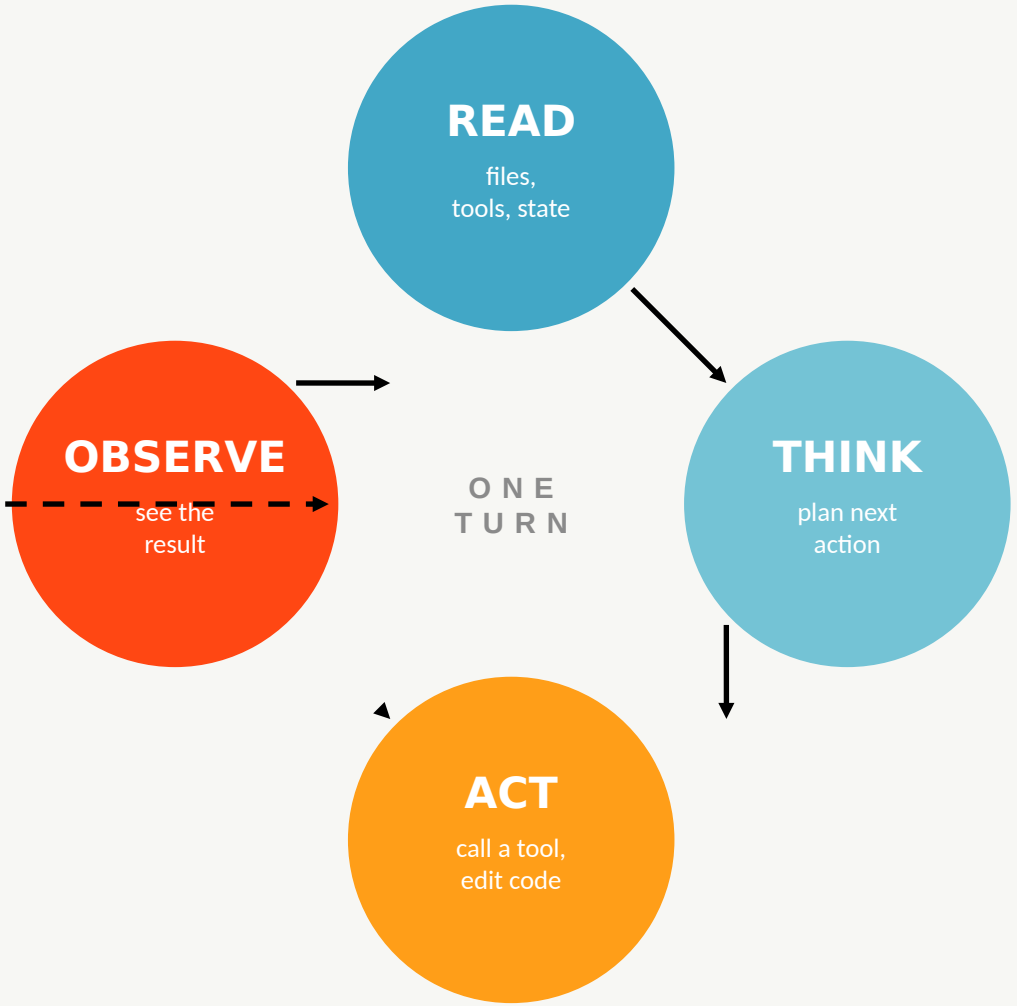
Shapes to your repo

CLAUDE.md teaches it your project. Skills give it playbooks. Rules constrain it. Hooks gate it. MCP gives it new tools. All per-repo, all version-controlled.

The agentic loop.

USER PROMPT

The trigger. Sometimes new — you type. Sometimes the loop keeps going on its own and only comes back when the agent gets stuck or finishes.



TODAY'S WORKSHOP RUNS ON OPUS 4.7

Opus in 2026.

Nov 2025

OPUS 4.5

The coding breakthrough

First Opus 4.x. Best model in the world for coding, agents, computer use when it landed. Handled ambiguity without hand-holding.

Feb 5, 2026

OPUS 4.6

Agent team + docs

Multi-agent orchestration. First-class PowerPoint generation. Better document workflows. Jumped ahead on long-horizon tasks.

Apr 16, 2026

OPUS 4.7

What we're using today

1M context at standard pricing. High-res vision (2576px). New tokenizer. Major gains on the hardest software engineering tasks — which is what a trading bot is.

← WORKSHOP

PLAN MODE — Shift+Tab twice. Even on Opus 4.7, make it plan before it touches files. That's how you stay the driver.

CLAUDE.md

When Claude Code starts in a directory, the first thing it reads is CLAUDE.md. That file tells it what the project is, how the code is organized, what conventions to follow, what to never do.

The app-in-a-box repo ships with a detailed CLAUDE.md — that's why, when you run 'claude' in the repo later, it already knows the bot's architecture, the git workflow, the service-layer pattern, and that you're building a trading bot for Bots & Bytes.

You'll write CLAUDE.md files for your own projects. Think of it as onboarding a new teammate. Short, specific, load-bearing.

THE HABIT THAT CATCHES MISTAKES EARLY

Read. The. Tool. Calls.

```
> build me a momentum strategy for ETH on 1h

● Using tool: search_reference_strategies
    asset: "ETH"
    timeframe: "1h"
    goal_hint: "momentum"

  Returned: 3 candidates (ref-001, ref-004, ref-007)

● Using tool: build_strategy_from_reference
    reference_id: "ref-001"

  Returned: strategy payload (signals, timeframe, risk)

● Using tool: create_strategy_manual
    ...
```

Every call is named, the arguments are visible, the result is visible. If the agent did something you didn't expect, the log tells you exactly what.

Four ways to shape behavior.

SKILLS

Step-by-step playbooks

When you say 'I want to create a strategy,' Claude loads .claude/skills/create-strategy/SKILL.md and follows it. Deterministic recipes the agent invokes on demand.

RULES

Global guardrails

Loaded every session start. .claude/rules/trading-bot-workflow.md is why this bot always tries a strategy before a manual swap. Shape how, not what.

HOOKS

Intercept scripts

Run before/after prompts or tool calls. block-wallet-secrets.sh scans every message you send for private-key patterns and refuses if it finds one. The safety you can't skip.

MCP

Connect external tools

Model Context Protocol. Claude Code doesn't know about trading — MCP servers hand it tools. This repo exposes 41 trading tools via a local MCP server.

Mangrove®

COMPANION TO BOTS & BYTES

Skills · Hooks · MCP Servers · Plugins — what they are, why they matter, how to build your first one.

APRIL 2026 · MANGROVE

Four primitives. One agent.

The things you can add to Claude Code to shape its behavior. Each one solves a different problem; together they cover the whole surface from 'teach it a new capability' to 'gate it from doing something dangerous.'

01 · SKILLS

Teach it

Reusable playbooks Claude loads on demand. 'When you're doing X, follow these steps, use these templates.' Extends WHAT Claude knows how to do.

02 · HOOKS

Gate it

Scripts that fire on agent events — before a tool call, after a file edit, on prompt submit. Extends WHEN and WHETHER Claude acts.

03 · MCP

Give tools

Standard protocol for plugging in external systems — APIs, databases, services. Extends WHAT Claude can touch beyond your files.

04 · PLUGINS

Ship it

Bundles of skills, hooks, MCP servers, and commands — installable as one package. Extends HOW you distribute all of the above.

PRIMITIVE 01

Teach Claude a new capability.

Modular playbooks Claude auto-loads when the task calls for them. The answer to 'how do I make it do X the same way every time?'

Skills.

WHAT IT IS

A folder with a SKILL.md file plus optional supporting files (scripts, templates, references). The SKILL.md has YAML frontmatter (name, description, when-to-trigger) and markdown instructions Claude follows when invoked.

Filesystem-based. No API upload. Version-controlled alongside your code.

WHY YOU'D USE ONE

You have a workflow you repeat: producing a doc in a specific format, running a recipe against data, authoring code that matches a house style. Writing it down once as a skill means Claude does it the same way every time — and you stop re-explaining.

Think of it as onboarding docs the agent can actually use.

How skills work.

STEP 01

DISCOVERY

Claude scans .claude/skills/ (and user-level dirs) at session start. Reads only the frontmatter — name + description — of each SKILL.md. Cheap. Builds a catalog.

→

STEP 02

TRIGGER

When your prompt matches a skill's description — e.g., 'make me a spreadsheet' matches xlsx skill — Claude loads the full SKILL.md body into its context.

→

STEP 03

EXECUTE

Claude follows the skill's instructions. If the skill references supporting files (scripts, templates), it reads those too. You see normal tool calls.

This is progressive disclosure. *The full instructions stay out of Claude's context until needed. Cheap to have hundreds of skills, expensive to load them all at once.*

```
// .claude/skills/create-strategy/SKILL.md
---
name: create-strategy
description: Use when the user wants to build a trading
  strategy. Searches reference library, builds payload,
  backtests, evaluates verdict.
allowed-tools: [search_reference_strategies, create_strategy_manual,
  backtest_strategy, evaluate_strategy]
---

# Create Strategy

When the user asks to build a strategy:

1. Parse their request for asset, timeframe, goal_hint.
2. Call `search_reference_strategies` with those params.
3. Present the top 2-3 candidates. Wait for user pick.
4. Call `build_strategy_from_reference(reference_id=...)`.
5. Persist with `create_strategy_manual(...)`.
6. Immediately call `backtest_strategy` with mode="full".
7. Present verdict using thresholds in ./threshold_spec.json.
```

Learn more about skills.

OFFICIAL REFERENCE

Extend Claude with skills — Claude Code Docs

https://code.claude.com/docs/en/skills

CONCEPTS

Agent Skills — Overview

https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview

BEST PRACTICES

Skill authoring best practices

https://platform.claude.com/docs/en/agents-and-tools/agent-skills/best-practices

TUTORIAL

How to create custom Skills

https://support.claude.com/en/articles/12512198-how-to-create-custom-skills

PRIMITIVE 02

Intercept. Gate. Refuse.

Shell commands that fire on agent lifecycle events. The answer to 'how do I stop the agent from ever doing X?' or 'how do I run Y every time it does Z?'

Hooks.

WHAT IT IS

A shell command (or script) that runs automatically when a specific Claude Code event fires. Configured in settings.json via event name, matcher (which tool triggers it), and the command to run.

Hooks execute in YOUR local shell, with YOUR user permissions. They communicate back via exit codes and stdout — exit non-zero to block an action.

WHY YOU'D USE ONE

Agents are autonomous. You can't be in the loop on every tool call. Hooks are the automated safety net:

- Scan prompts for secrets before they reach the model.
- Refuse writes to protected paths.
- Log every tool call to an audit file.
- Run formatters / linters after edits.

Not 'please be careful' — structural refusals.

The eight lifecycle events.

ONCE PER SESSION

SessionStart

Fires when Claude Code starts in a project. Load context, check env, warn on drift.

SessionEnd

Fires when the session terminates. Flush logs, commit state, notify.

ONCE PER TURN

UserPromptSubmit

Fires on every message you send. Scan for secrets, rewrite, or refuse.

Stop

Fires when Claude finishes responding. Final checks, summaries.

StopFailure

Fires when Claude's response fails. Error handling, retries.

ON EVERY TOOL CALL

PreToolUse

Before a tool runs. Inspect args, block dangerous calls, rewrite params.

PostToolUse

After a tool returns. Log, audit, trigger side effects (format, test).

Exit code 0 = allow. Non-zero = block. stdout can rewrite the prompt or inject instructions.

```
// .claude/settings.json
{
  "hooks": {
    "UserPromptSubmit": [{
      "command": "./.claude/hooks/block-wallet-secrets.sh"
    }]
  }
}

// .claude/hooks/block-wallet-secrets.sh
#!/bin/bash
input=$(cat)
if echo "$input" | grep -qE '[0-9a-fA-F]{64}|\b([a-z]+ ){11,}[a-z]+\b'; then
  echo "Blocked: looks like a private key or seed phrase." >&2
  echo "Use ./scripts/stash-secret.sh instead." >&2
  exit 1    # non-zero exit = refuse the prompt
fi
echo "$input"  # pass through unchanged
```

This is Safety Net 1 from the Bots & Bytes bot. Your prompt never reaches the model if it matches a key pattern.

Learn more about hooks.

OFFICIAL REFERENCE

Hooks reference — Claude Code Docs

https://code.claude.com/docs/en/hooks

GUIDE

Automate workflows with hooks

https://code.claude.com/docs/en/hooks-guide

BLOG / WALKTHROUGH

How to configure hooks — Claude power-user customization

https://claude.com/blog/how-to-configure-hooks

SDK

Intercept and control agent behavior with hooks

https://platform.claude.com/docs/en/agent-sdk/hooks

PRIMITIVE 03

Give Claude new tools.

Open-source standard for connecting AI agents to external systems. The answer to 'how do I let Claude talk to my database / API / service?'

MCP servers.

WHAT IT IS

A protocol standard for AI-to-tool integrations, plus the servers that implement it. An MCP server exposes a set of named tools (e.g., `create_wallet`, `get_market_data`) — Claude Code connects to it and discovers the tools at session start.

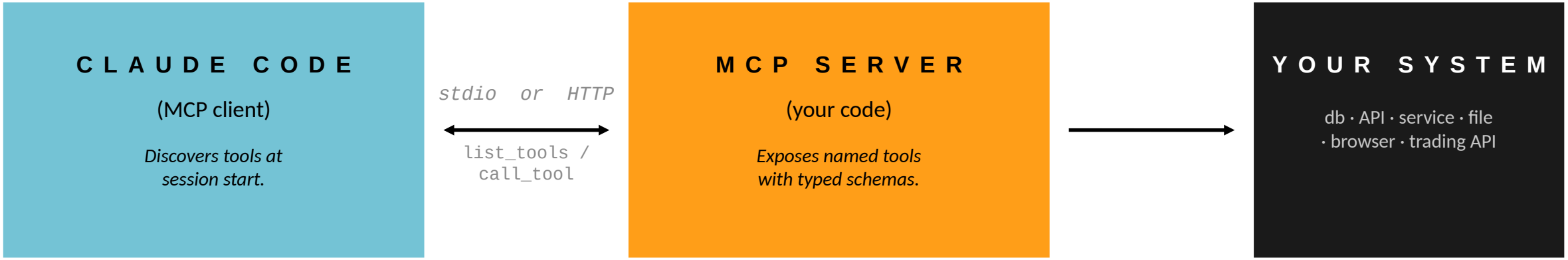
Servers can run locally (process on your machine) or remotely (HTTP). Built by Anthropic, but truly open — works with many AI clients, not just Claude.

WHY YOU'D USE ONE

Without MCP, Claude Code only knows about files and your shell. With MCP, it can query a database, hit an internal API, read from your email, place a trade — anything you expose through a server.

And because the protocol is standard, the server you build for Claude works with every other MCP-compatible client.

Client, server, transport.



Discovery first, then call. At session start, Claude asks the server 'what tools do you have?' and gets back names + schemas. Tool calls are just named, typed invocations of what was advertised.

```
# Register a local MCP server with Claude Code
$ claude mcp add -s local -t http defi-agent \
    http://localhost:9080/mcp/ \
    --header "X-API-Key: dev-key-1"

# List what's connected
$ claude mcp list
defi-agent:  http://localhost:9080/mcp/  (HTTP)  - ✓ Connected

# Start a session — tools appear automatically
$ claude

> list my tools

● Using tool: list_tools
  Returned: 41 tools
  status, create_wallet, list_wallets, get_portfolio, ...
  create_strategy_manual, backtest_strategy, ...
  execute_swap, get_market_data, kb_search, ...
```

One register command, 41 new tools. This is how the Bots & Bytes bot gets trading capabilities Claude Code doesn't have out of the box.

Learn more about MCP.

CLAUDE CODE MCP

Connect Claude Code to tools via MCP

https://docs.anthropic.com/en/docs/claude-code/mcp

ANNOUNCEMENT

Introducing the Model Context Protocol

https://www.anthropic.com/news/model-context-protocol

QUICKSTART

Build your first MCP server — tutorial

https://modelcontextprotocol.io/quickstart

REFERENCE

Model Context Protocol — Claude Docs

https://docs.claude.com/en/docs/mcp

PRIMITIVE 04

Ship all the above as one package.

Plugins bundle skills, hooks, MCP servers, and slash commands into an installable unit. The answer to 'how do I share my setup with my team?'

Plugins.

WHAT IT IS

An installable bundle that packages any combination of skills, hooks, MCP servers, agents, and slash commands — with a manifest describing what's inside.

Distributed through marketplaces (catalogs of plugins). Anthropic maintains an official marketplace, enterprises can run private ones, and you can pull plugins from any GitHub repo.

WHY YOU'D USE ONE

You've built 12 custom skills, 4 hooks, and an MCP server for your team's workflow. Without plugins, every new engineer rebuilds that setup from scratch.

With plugins: one install command, everyone has the same Claude Code environment. Company-wide coding conventions, safety policies, and tool integrations — distributed as a single package.

From marketplace to installed.

STEP 01

MARKETPLACE

A catalog (official Anthropic marketplace, a private company one, or any GitHub repo). Run /plugin inside Claude Code to browse the Discover tab.

→

STEP 02

INSTALL

Pick a plugin, /plugin install. Claude Code downloads it, validates the manifest, and wires up the skills, hooks, MCP servers, commands it declares.

→

STEP 03

ACTIVE

The plugin's primitives are now part of your session. Skills auto-trigger. Hooks fire on events. MCP tools appear. Commands are callable.

→

STEP 04

UPDATES

Run /plugin update. Pulls the latest version from the marketplace. Audit trail tells you what changed.

Enterprises: *private marketplaces for company-approved plugins. Cross-compatible with Cowork + Agent SDK.*

```
# Browse the official marketplace
> /plugin

  claude-plugins-official
    anthropic-skills     Reference skills bundle
    mcp-builder          Scaffold new MCP servers
    skill-creator        Guided skill authoring
    theme-factory        Style artifacts with themes

# Install one
> /plugin install anthropic-skills
✓ Installed. 10 skills loaded.

# Register a private marketplace (your org's)
$ claude plugin marketplace add mangrove/bots-bytes-plugins
✓ Added. 3 plugins available.
```

The /plugin command opens the browser for the official marketplace. Private orgs can point Claude Code at their own GitHub-hosted catalog.

Learn more about plugins.

OFFICIAL REFERENCE

Discover and install plugins through marketplaces

https://code.claude.com/docs/en/discover-plugins

BLOG

Customize Claude Code with plugins

https://claude.com/blog/claude-code-plugins

CATALOG

Plugins for Claude Code and Cowork

https://claude.com/plugins

ENTERPRISE

Cowork and plugins for teams across the enterprise

https://claude.com/blog/cowork-plugins-across-enterprise

How they compose.

PLUGIN	*the distributable wrapper*
SKILLS	*playbooks — what Claude knows how to do*
MCP SERVERS	*tools — what Claude can touch*
HOOKS	*gates — when Claude may act*
SLASH COMMANDS	*shortcuts users invoke directly*

Mangrove®

BEFORE WE TOUCH REAL MONEY

Leverage multiplies what you do right. **It also multiplies what you do wrong.**

In coding: data loss, false success, leaked secrets.

In finance: money gone. Not recoverable.

Everything after this point can move real funds. Ground rules first.

FROM THE TRENCHES

War stories.

Four categories of things agentic AI gets wrong when you're not watching. Real scenarios — not theoretical.

CATEGORY 01

Data loss

"It overwrote the file I'd just saved."

Edits without reading. Forgets what's on disk. Happens most when context is thin or plan mode was skipped.

CATEGORY 02

False success

"It said the tests passed. They didn't."

Fabricated results. Claimed completion it didn't achieve. The fix: read the tool calls, not the summary.

CATEGORY 03

Prompt injection

"It followed instructions it read off the internet."

A README, a GitHub issue, a web page — anything the agent reads can contain 'now do X.' Trust boundary breach.

CATEGORY 04

Runaway autonomy

"It kept going when it should have stopped."

Bypass permissions, no plan mode, no hooks. Burned tokens, made mess, sometimes destroyed state.

NAMING THE THING

What is agentic finance?

AI agents that **autonomously execute financial actions**. Not rule-based bots. Not AI advisors. Agents that reason, decide, and act — with your money, your keys, and on-chain state. Usually built on top of LLM-driven reasoning + tool use.

LEVEL 01

Trading bots

What we're about to build. Agent reads market data, evaluates strategies, executes swaps. Narrow mandate, explicit allocation, human-set strategy. The entry-level of the category.

LEVEL 02

DeFi autopilots

Agent-managed yield strategies, auto-rebalancing portfolios, leverage managers. Multi-protocol, multi-asset, responding to signals continuously. Delegated authority, scoped actions.

LEVEL 03

Autonomous treasuries

An agent manages an org's operating capital — payroll, vendor payments, hedging, treasury allocation. Still theoretical at scale; first serious deployments happening now. The end of the runway.

We're doing Level 01 today. Everything above it builds on these same primitives.

What changes when finance gets agentic.

WHAT OPENS UP

Democratization. Institutional-grade strategies — backtesting, risk models, multi-factor signals — become accessible to anyone who can prompt an agent.

Speed. Agents react in milliseconds, 24/7, without emotion. Strategies that required a desk of traders can run on one laptop.

Personalization. Every user gets their own agent, tuned to their risk, their time horizon, their values. The 'one-size-fits-millions' fund model gets competition.

Composability. Agents that talk to agents — settling trades, negotiating pricing, paying each other. The machine-readable economy.

WHAT'S NEW AT RISK

Correlated failure. Agents trained on similar data act similarly in crises. Flash crashes amplified. Herd behavior, but in silicon.

Attribution. When an agent loses your money, who's responsible? The model? The operator? The strategy author? Law hasn't caught up.

New attack surfaces. Prompt injection. Data poisoning. Agent hijacking. Problems traditional security didn't have to think about.

Regulatory arbitrage. First movers set norms before regulators arrive. That's opportunity — and it's also how things break at scale.

YOUR NEW ATTACK SURFACE

THE OLD PARADIGM

Keep them out.

Threat model: unauthorized code execution, unauthorized access.

Attack vectors: SQL injection · XSS · RCE · credential theft · malware.

Defenses: firewalls · auth · input validation · WAFs · EDR. Deterministic. Auditable. Decades of tooling.

THE NEW PARADIGM

Stop authorized agents from being manipulated.

Threat model: your agent, given legitimate access, is talked into doing the wrong thing.

Attack vectors: prompt injection (direct + indirect) · data poisoning · adversarial content · supply-chain compromise on model weights.

Defenses: hooks · permission prompts · least-privilege scopes · treat-inputs-as-hostile. Non-deterministic. Still being invented.

Prompt injection is the new SQL injection. Your data is your code. A README can pwn you.

PUTTING THIS TO WORK

You're about to do this.

What comes next is agentic finance in miniature. You're going to create a wallet, fund it with real money, and hand the keys to an AI that will trade on your behalf.

The four rules from two slides ago aren't abstractions — they're what keeps this exercise safe. **Every guardrail you'll see in the next section is a worked example of those rules:**

RULE →

Read the tool calls

GUARDRAIL

`list_trades · list_evaluations`

RULE →

Version control

GUARDRAIL

`agent.db as audit trail`

RULE →

Treat inputs as hostile

GUARDRAIL

`key-paste hook refuses secrets`

RULE →

Budget your autonomy

GUARDRAIL

`backup gate + slippage cap`

Let's set up the wallet.

"The same thing that lets Claude build your app in 30 seconds can wire your API key to a bad endpoint if you let it."

— A NOTE BEFORE WE BUILD ANYTHING

Two exhibits.

EXHIBIT A

"Enable bypass permissions mode."

Audience: 600,000 subscribers.

WHAT'S BROKEN Skips every permission prompt. The prompts ARE the safety layer.

WHY IT MATTERS An agent touching real keys or endpoints, with no user-in-the-loop, can be weaponized by a prompt-injection in any README, issue, or web page it reads.

EXHIBIT B

"Paste your API key. Store it in .env."

Audience: a general crypto crowd.

WHAT'S BROKEN Keys in chat end up in transcripts, logs, and exports. .env files get committed to public repos constantly — GitHub scrapers find them in minutes.

WHY IT MATTERS You can't un-leak a key. You can only rotate. Scoped keys, secret managers, pre-commit hooks are the actual answer.

"Convenience is a security budget. Know what you're spending it on."

— THE PRINCIPLE WE'RE GOING TO KEEP COMING BACK TO

BUILT INTO THIS BOT

The three safety nets.

NET 01

Key-paste block

A hook scans every prompt you submit. If it matches a private key or seed phrase shape, your message never leaves the terminal. Not the bot, not the logs, not the transcript.

NET 02

Backup gate

Every wallet has a backup_confirmed_at column. Live trading refuses to run until you've saved the secret somewhere durable. and explicitly confirmed with a terminal command. The secret is NOT stored in plaintext, is never revealed in a chat session, never sent to anthropic, and never crosses the internet. If you don't save it, you risk losing everything.

NET 03

Keys never in plaintext

Wallet keys are encrypted at rest with Fernet. Master key lives in your OS keychain, or a chmod-600 file if you opt out. The agent decrypts in-memory only to sign, then wipes. A stolen laptop or a disk leak doesn't leak keys.

The key-paste block.

THE PROBLEM

You fat-finger your private key into chat. The bot sees it. Anthropic's logs see it. Your ~/.claude/projects/<hash>/ transcript file sees it. If you ever share the transcript, the key goes with it.

You can't un-leak a key. You can only rotate.

THE NET

`.claude/hooks/block-wallet-secrets.sh`

Runs on every prompt you submit. Scans for 64-char hex patterns and common mnemonic shapes. Match = refusal, before the message leaves your terminal.

To actually import a wallet, use: `./scripts/stash-secret.sh`

— *hidden-input prompt, local-only, never touches the chat.*

The backup gate.

STEP 01

CREATE

create_wallet
returns secret_id

backup_confirmed_at: NULL

Live trading: REFUSED

→

STEP 02

REVEAL

./scripts/reveal-secret.sh

One-shot terminal output.
Prints the key, doesn't log it.

Live trading: REFUSED

→

STEP 03

CONFIRM

./scripts/confirm-backup.sh

You've saved it somewhere durable
— password manager, paper, hw wallet.

Live trading: UNLOCKED

Four requirements for any live fire: user asks · wallet confirmed · allocation complete · confirm=true. All four or nothing happens.

Keys never in plaintext.

Every wallet key is **Fernet-encrypted** before it hits disk. What's stored in `agent.db` is ciphertext — useless without the master key.

The master key lives in your **OS keychain** (macOS Keychain / Windows Credential Manager / Linux Secret Service). If you opt out, it's a file at `agent-data/master.key`, chmod 600.

The agent decrypts **in-memory only to sign**, wipes the plaintext immediately after. A disk leak, a stolen laptop, a stray backup — the key stays sealed.

DEFENSE IN DEPTH

NET 01
Chat layer — can't paste keys

NET 02
Authorization — gated live fire

NET 03
At rest — encrypted, keychain-sealed

THE ONE-LINE PHILOSOPHY

Paper before live. Always.

Every strategy starts in draft or paper. Promotion to live is a separate, explicit step — four requirements, all or nothing. We stop at paper today. Chapters 06–08 take you the rest of the way, on your own time, when you're ready.

ACT 02

WHAT YOU HAVE.

Architecture. Trust boundaries. Where the bot ends and the world begins.

THE WHOLE PICTURE

Where things run.

YOUR LAPTOP · TRUST BOUNDARY

CLAUDE CODE

(terminal)

`~/.claude`

MCP over HTTP →

`:9080`

DEFI-AGENT · UVICORN :9080

- 41 MCP tools + REST API
- APScheduler (in-process cron)
- Fernet-encrypted wallet keys
- SQLite: strategies, trades, wallets

agent-data/

`agent.db`
`master.key`

MANGROVE · HOSTED API

Signals · strategies · knowledge base · OHLCV

Never sees your private key.

BASE MAINNET · PUBLIC CHAIN

1inch router · USDC · ETH · WBTC

Only touched when you go LIVE.

THE THING WORTH INTERNALIZING

Your laptop is the trust boundary.

Keys live here. Trades execute from here. State lives here. Anything beyond the laptop is either public (on-chain) or an advisor (Mangrove) — neither can move your funds. If Mangrove is breached tomorrow, you still have your wallet.

ONE PYTHON PROCESS, PORT 9080

The defi-agent.

It serves a REST API under /api/v1/agent/* and an MCP endpoint at /mcp/ — Claude Code talks to the MCP side. Same service layer behind both. 41 tools.

It runs the scheduler in-process (APScheduler). When a strategy is paper or live, the scheduler registers a cron job that fires evaluate_strategy on the timeframe. No separate daemon, no system cron.

It holds wallet keys Fernet-encrypted in the SQLite file. The master key lives either in your OS keychain or a 600-perm file at agent-data/master.key. The agent decrypts only in memory, only to sign, wipes immediately after.

If this process isn't running, nothing works. One `ps aux | grep uvicorn` check tells you everything.

The hosted side.

mangroveai

Signals + strategies

Your bot doesn't reimplement signal math. It sends strategy + market snapshot, gets back order intents. RSI, MACD, ichimoku, ADX, all of it.

mangrove-kb

The knowledge base

Curated library of signals and patterns. Every parameter choice cites a KB entry — no vibes. Try kb_search("momentum") later.

mangrovemarkets

Market data + DEX routing

OHLCV, spot prices, gas, 1inch quotes. Prepares the unsigned swap payload. The agent signs it locally and hands the signed tx back for broadcast.

"Nothing touches Base mainnet until YOU go live. Paper mode is a ghost — identical in your DB, zero on-chain, zero funds."

— THE DEFAULT IS SAFE

Mangrove®

INSIDE MANGROVE

The open infrastructure your trading bot is built on.

THE STACK

Three layers, one bot.

When your bot asked for a momentum strategy, three pieces of Mangrove infrastructure cooperated to answer. Here's what they are and what each one does.

LAYER 01

Mangrove KB

mangrove-kb

Open-source library of trading signals and technical indicators. 223 signals, 40+ indicators, all registered via a Python decorator pattern. Runs locally; no API call needed.

LAYER 02

MangroveAI SDK

mangroveai

Python client for Mangrove's hosted intelligence — strategy authoring, backtesting, signal evaluation, reference strategies, on-chain queries, DeFi data, social signals.

LAYER 03

MangroveMarkets SDK

mangrovemarkets

Python client for Mangrove's execution layer — market data, wallet management, DEX routing via 1inch. The thing that actually signs transactions and broadcasts to Base.

K N O W L E D G E B A S E

Open-source library of trading signals and indicators. Everything the bot composes strategies from.

What the KB is.

A pip-installable Python package — `mangrove-kb` — containing 223 trading signals and 40+ technical indicators, all implemented as composable Python functions. Open source. Runs locally. No API key, no network call.

Every signal is a pure function: **it takes a pandas DataFrame of OHLCV data and returns a boolean.**

Every indicator is a class implementing `IndicatorInterface` — computes a derived time series (RSI, MACD line, Bollinger bands, etc.). Signals consume indicators.

OPEN

MIT-style. Read it, fork it, extend it, audit it.

LOCAL

Runs in-process. No API call, no latency, no rate limit.

COMPOSABLE

Functions you combine into strategies with AND / OR / NOT logic.

223 signals. 40+ indicators.

Coverage across every major technical analysis family. Growing.

TREND	**88** signals	SMA · EMA · MACD · ADX · Aroon · Ichimoku · PSAR · DEMA · ALMA
MOMENTUM	**42** signals	RSI · Stochastic · Williams %R · KAMA · ROC · PPO · StochRSI
PATTERNS	**40** signals	Engulfing · Doji · Harami · Pin bar · Evening star · Dark cloud cover
VOLUME	**33** signals	OBV · MFI · CMF · VWAP · ADI · ADOSC · Ease of movement
VOLATILITY	**20** signals	Bollinger Bands · ATR · Keltner Channel · Donchian · Chandelier exit

```python
// mangrove_kb/signals/momentum.py

from mangrove_kb.registry import RuleRegistry
from mangrove_kb.indicators import RSI
import pandas as pd

@RuleRegistry.register("rsi_overbought")
def rsi_overbought(df: pd.DataFrame, window: int = 14,
                   threshold: float = 70.0) -> bool:
    """
    Check if RSI is above the overbought threshold.
    Type: FILTER. Requires: Close.
    """
    rsi = RSI(df, window=window).compute()
    return rsi.iloc[-1] > threshold

# Now addressable by name anywhere in the stack:
# RuleRegistry.evaluate({"name": "rsi_overbought"}, df)
```

Pure function + decorator = named signal. Add one signal = four lines. 223 of these compose every strategy the bot will ever build.

Signals come with context.

Every entry is a Python function with a real docstring — what it detects, when it fires, and crypto-specific caveats. Here are three from different categories, verbatim from the library.

TREND

macd_bullish_cross

DETECTS MACD line crosses above the signal line.

CONTEXT A bullish MACD crossover occurs when the MACD line crosses above the signal line, indicating potential upward momentum.

NOTE *Crypto's high volatility may produce frequent signals; use with trend confirmation.*

PATTERNS

bullish_engulfing_trigger

DETECTS Current bullish candle's body completely contains the previous bearish candle's body.

CONTEXT Strong bullish reversal pattern. One of the most reliable price-action signals when it appears at key support levels.

NOTE *References: [NISON], [KB-07], [STOCKCHARTS].*

VOLATILITY

bb_squeeze

DETECTS Bollinger Band width drops below a volatility threshold.

CONTEXT A squeeze signals low volatility and an impending breakout. Traders use it to identify coiled setups before directional moves.

NOTE *Fires ONCE on the onset bar, not while the squeeze persists.*

```
> Build me a momentum strategy for ETH on 1h.

● Using tool: kb_search
    query: "momentum"
    returns: 42 signals — rsi_cross_up, sma_cross_up,
             macd_bullish_cross, stoch_cross_up, ...

● Using tool: signals.get
    signal_name: "rsi_cross_up"
    returns: {
      description: "Check if RSI crosses above a threshold.
        In crypto, consider higher thresholds (80/20) during
        strong trends.",
      default_params: {window: 14, threshold: 30}
    }

  Proposing strategy — each signal cited by name:
    entry:  [rsi_cross_up, sma_cross_up, adx_strong_trend]
    exit:   [rsi_cross_down, sma_cross_down]

  (At runtime, RuleRegistry.evaluate() dispatches by name.)
```

Four-stage pattern: Discover (kb_search) → Read (signals.get) → Cite (strategy dict by name) → Invoke (RuleRegistry dispatch). No prompt engineering substitutes for real curated content.

THE SDKs

Two Python clients. One for intelligence (MangroveAI), one for execution (MangroveMarkets).

MangroveAI SDK.

`pip install mangroveai` — the hosted-intelligence client. Strategy authoring, backtesting, signal evaluation, reference library, on-chain queries. Everything that reasons ABOUT markets, not in them.

`.auth`

API-key lifecycle

`.strategies`

Create, list, update, archive

`.backtesting`

Historical eval, 6-metric verdict

`.signals`

Evaluate KB signals against live data

`.execution`

Run a strategy, get order intents

`.kb`

Search reference strategies + docs

`.crypto_assets`

Asset metadata, contract addresses

`.on_chain`

Block state, tx lookups, gas

`.defi`

TVL, pool stats, protocol health

`.social`

Sentiment, mentions, trend data

```
from mangroveai import MangroveAI
client = MangroveAI(api_key="prod_...")
strategies = client.strategies.list()
```

MangroveMarkets SDK.

`pip install mangrovemarkets` — the execution client. Market data, wallet management, DEX routing. This is the layer that actually signs transactions and broadcasts to chain.

.wallet

Create / import wallets. Fernet-encrypted key storage. Balance queries. Self-custody by design — the SDK never exports your plaintext key.

.dex

Quote, approve, prepare swap, broadcast. Routes via 1inch. All transactions signed locally by YOUR process, not on Mangrove's servers.

.portfolio

Aggregated balances across chains. Historical P&L. Exposure by token, by position. The read-only view into what you own.

```
from mangrovemarkets import MangroveMarkets
client = MangroveMarkets(base_url="...")
quote = client.dex.get_quote(from_token="USDC", to_token="ETH", amount=5)
```

How the pieces fit.

MANGROVE KB

local, open source

Signal definitions.
Indicator classes.
Registry lookups.

→

MANGROVEAI SDK

hosted intelligence

Strategy evaluation,
backtesting, KB search,
on-chain + defi data.

→

MANGROVEMARKETS

hosted execution

Wallet custody, DEX
quotes, signed-tx broadcast,
portfolio state.

→

YOUR APP

defi-agent process,
scheduler, CLAUDE.md,
MCP server, audit trail.

KB ships the primitives. AI SDK composes and evaluates them. Markets SDK executes the intent. Your app orchestrates the loop.

SIGNAL LIBRARY

How you actually use 223 signals to build a strategy that trades.

```
// A strategy is a dict of signal rules — no imperative code needed.

strategy = {
    "name": "ETH momentum with RSI + SMA crossover",
    "asset": "ETH",
    "timeframe": "1h",
    "entry_rules": [
        {"name": "rsi_cross_up",       "params": {"window": 14, "threshold": 30}},
        {"name": "sma_cross_up",       "params": {"window_fast": 20, "window_slow": 50}},
        {"name": "adx_strong_trend",   "params": {"window": 14, "threshold": 25}},
    ],
    "exit_rules": [
        {"name": "rsi_cross_down",     "params": {"window": 14, "threshold": 70}},
        {"name": "sma_cross_down",     "params": {"window_fast": 20, "window_slow": 50}},
    ],
    "risk": {"stop_loss_pct": 0.04, "take_profit_pct": 0.08},
}
```

Five signals by name — Mangrove's KB + AI SDK do the math. No numpy, no TA-Lib, no wheel reinvention.

Search → compose → backtest → iterate.

STEP 01

SEARCH

kb_search("momentum") surfaces signals by theme. search_reference_strategies() finds pre-backtested combinations that match your asset + timeframe + goal.

→

STEP 02

COMPOSE

Build the strategy dict — entry rules, exit rules, risk config. Either dictate manually, let Claude Code author it from a reference, or run autonomous mode to generate candidates.

→

STEP 03

BACKTEST

backtest_strategy() runs your logic against historical OHLCV. Returns 6 metrics (Sortino, Sharpe, Calmar, IRR, max drawdown, win rate) + a PASS / MARGINAL / FAIL verdict.

→

STEP 04

ITERATE

Swap a signal, adjust a threshold, change timeframe, rerun. Seconds per iteration, not hours. The speed of this loop is the reason you use an AI agent to drive it.

Loops back from step 4 to step 1. Every iteration makes the strategy cheaper to test than the last.

BUILDING IN PUBLIC

Why Mangrove's foundation is open source — and what that means for you.

Open source is good for the community.

Trading infrastructure is one of the most consequential places to be careful about trust. OSS is how Mangrove earns it — and how the community gets something better in return.

01 Inspectable by default

Every signal in the KB is 10–40 lines of Python you can read, audit, and reason about. If you don't understand what it does, you can find out — no docs required, the code IS the docs.

02 Forkable when you disagree

If your read of a signal differs from ours, fork it. Change the threshold. Add a guard. Contribute it back if it's better. The barrier to disagreeing productively is one PR.

03 Shared by everyone who uses it

Every user who catches a bug or adds a signal helps every other user. The library gets better in proportion to how much it gets used — a feedback loop you can't buy.

04 Portable across tools

mangrove-kb doesn't care whether you're using Mangrove's hosted API, your own backtester, or a Jupyter notebook. Open primitives travel with you even if the platform doesn't.

Open primitives. Hosted intelligence.

OPEN SOURCE

`mangrove-kb`
The signal library. 223 signals, 40+ indicators. Pip install, fork, PR against it.

`SDK clients`
mangroveai and mangrovemarkets Python clients. Open so you can audit what gets sent over the wire.

`Templates`
app-in-a-box (this workshop's repo), x402 templates, trader plugins. Fork and build.

HOSTED SERVICE

Strategy evaluation + backtesting
The compute-heavy path. OHLCV streaming, parallel backtest, risk metrics.

Reference strategy library
Curated strategies with vetted backtests. Growing as users contribute via API.

DEX routing + on-chain services
1inch integration, gas estimation, tx prep. Never sees your private key — you sign locally.

CONTRIBUTE: `github.com/MangroveTechnologies` · PACKAGES: `pypi.org/project/mangrove-kb` · `mangroveai` · `mangrovemarkets`

Mangrove®

MANGROVE INTERNALS · RESOURCES

GITHUB: github.com/MangroveTechnologies
DOCS: mangrovedeveloper.ai
PYPI: mangrove-kb · mangroveai · mangrovemarkets
FEEDBACK: tim.darrah@mangrove.ai

BUILT IN PUBLIC. LIBRARY GROWS WITH YOU.

HALFWAY

Break.

10 minutes. Hydrate. Ask your neighbor a question. Check your bot is still running.

ACT 03

YOUR FIRST STRATEGY.

From empty database to a backtested, committed strategy. Reference-first. PASS / MARGINAL / FAIL — and what to do about each.

THE FIVE PARTS

What a strategy is.

01

Asset

The token you want to trade. Workshop default: ETH.

02

Timeframe

Cadence for evaluation. 5m · 15m · 30m · 1h · 4h · 1d. We'll use 1h — enough signal, not too much noise, fast backtests.

03

Entry signals

Conditions that open a position. E.g. rsi_cross_up, sma_cross_up, adx_strong_trend.

04

Exit signals

Conditions that close. rsi_cross_down, trailing stop, fixed target.

THREE WAYS TO AUTHOR

Pick your authoring mode.

MODE	WHAT HAPPENS	WHEN TO USE
Reference-first	Agent searches a curated library for matches. Shows you top 2–3. You pick one. Signals are copied verbatim from a strategy that's already been backtested.	**First strategy. Always. This is the workshop default.**
Manual	You dictate the exact signals and parameters. The agent must cite a KB entry for every choice — no freestyling.	You've read the KB and know what you want.
Autonomous	Agent generates 7 candidate strategies itself, backtests all of them, ranks by performance. ~60 seconds vs ~10 for reference-first.	You want a second opinion, or couldn't find a good reference match.

Ground truth, fast.

Every reference strategy in the library has been backtested before it was added. You're not starting from zero — you're testing whether a proven structure still works on today's market.

The signal choices are fixed. Copied verbatim. So you're testing the market, not the strategy. That's a cleaner hypothesis.

It's fast. ~10 seconds for reference-first vs ~60 for autonomous. In a workshop, speed matters.

TYPE THIS VERBATIM

Your first prompt.

"Build me a momentum strategy for ETH on 1h. Use a reference."

What the bot will do:

1. search_reference_strategies(asset="ETH", timeframe="1h", goal_hint="momentum")
2. Return 2–3 ranked candidates with signal names + categories
3. Wait for you to pick one

WHAT YOU'LL SEE BACK

Reading the candidates.

```
Here are three momentum-oriented reference strategies for ETH on 1h:

1.  ref-001 — ETH momentum with RSI + SMA crossover
    Entry:  rsi_cross_up(14, 30), sma_cross_up(20, 50),
            adx_strong_trend(14, 25)
    Exit:   rsi_cross_down(14, 70), sma_cross_down(20, 50)

2.  ref-004 — ETH breakout on ADX + MACD
    Entry:  adx_strong_trend(14, 25),
            macd_bullish_cross(12, 26, 9)
    Exit:   macd_bearish_cross(12, 26, 9)

3.  ref-007 — ETH trend-following with ichimoku
    Entry:  ichimoku_bullish, sma_cross_up(20, 50)
    Exit:   ichimoku_bearish
```

Pick one. For the workshop: 'Let's try ref-001.' You can always change your mind later — strategies are cheap.

Six metrics. Six thresholds.

METRIC	THRESHOLD	WHAT IT MEANS
Sortino	≥ 1.5	Risk-adjusted return, downside-only. Higher is better.
Sharpe	≥ 1.2	Risk-adjusted return, total volatility. Higher is better.
Calmar	≥ 1.0	Return vs max drawdown. Higher is better.
IRR	≥ 0.15	Annualized return. 15% is the floor.
Max drawdown	≤ 0.70	Biggest peak-to-trough dip. Lower is better.
Win rate	≥ 0.25	Fraction of trades that were profitable.

VERDICT All 6 met = PASS · 4–5 met = MARGINAL · ≤ 3 = FAIL

What to do with each verdict.

PASS

Promote to paper

All 6 thresholds met. The strategy is a reasonable hypothesis. Promote to paper, watch it tick, refine later if needed. This is the happy path.

MARGINAL

Accept or iterate

4 or 5 of 6. The bot tells you which missed. You can paper it anyway (thresholds are conservative) OR ask for a different reference / different timeframe.

FAIL

Do NOT paper it

≤ 3 met. Market regime has moved against this strategy's logic. Papering a failing strategy to 'see what happens' just wastes cron ticks. Pick another reference.

"INSUFFICIENT_TRADES is not a failure. It's 'no evidence either way — try again with more data.'"

— THE SPECIAL CASE EVERY BEGINNER MISREADS

ACT 04

PAPER MODE.

Schedule it. Watch it tick. Simulated fills. No funds. No on-chain exposure. The last stop before live.

ONE SENTENCE, ONE TOOL CALL

Promote it to paper.

```
> Promote it to paper.

● Using tool: update_strategy_status
    strategy_id: "..."
    status: "paper"

Promoted to paper. Scheduler registered a cron job on the 1h
timeframe — evaluations will fire at the top of each hour.
Active cron jobs: 1.

Check `list_evaluations` any time to see what the strategy saw.
```

The scheduler uses the SAME SQLite file as everything else, so cron jobs survive server restarts. Your bot can reboot and the strategy picks up ticking.

CRON BY TIMEFRAME

When your strategy ticks.

TIMEFRAME	CRON EXPRESSION	MEANING
5m	`*/5 * * * *`	Every 5 minutes
15m	`*/15 * * * *`	Every 15 minutes
30m	`*/30 * * * *`	Every 30 minutes
1h	`0 * * * *`	Top of every hour ← workshop default
4h	`0 */4 * * *`	Every 4 hours
1d	`0 0 * * *`	Midnight UTC

DON'T WAIT AN HOUR

Force a tick.

```
> Run evaluate_strategy on it so I don't have to wait for the cron.

● Using tool: evaluate_strategy
    strategy_id: "..."

Evaluated. Duration: 487 ms. Orders produced: 1.

Paper trade logged:
  side:        buy
  symbol:      ETH
  input:       100 USDC
  output:      0.0398 ETH  (at fill_price $2,509.21)
  mode:        paper
  reason:      rsi_cross_up + sma_cross_up both fired
```

Same tick logic the cron would have run, right now. If orders == 0, the signal just isn't firing — that's not a malfunction, that's the strategy watching.

Evaluations vs trades.

list_evaluations

Every cron tick logs a row — whether or not the strategy fired. Status, duration, how many orders, and a one-line note on what the strategy saw.

Zero trades after 5 ticks? That's not a bug. The signals just haven't been met. Let it run longer.

Durations over 5 seconds? Mangrove is probably under load.

list_trades

Simulated fills in paper mode. Same schema as live trades — mode column is 'paper' instead of 'live', and tx_hash is null.

One row per fill. Side, symbol, input, output, status. Lets you verify the strategy's pattern (frequency, size, direction) against the exact surface you'd see in live mode.

Create or import.

CREATE A NEW WALLET

Ask the bot: "Create me a new wallet." It calls create_wallet, generates a fresh EVM keypair in-process, Fernet-encrypts the private key, stores it in agent.db, and returns you an address + secret_id.

Best path for a workshop: nothing to transfer, no existing keys to manage, clean audit trail from minute one.

IMPORT AN EXISTING WALLET

Run ./scripts/stash-secret.sh in your terminal. Hidden-input prompt accepts your private key, encrypts it, stores it. The bot never sees the plaintext.

Use when you already have a funded wallet you want the bot to drive. NEVER paste a key into chat — the hook will refuse it anyway.

The encryption model.

AT REST

Fernet ciphertext in SQLite

The private key is encrypted before it hits disk. What's in agent.db is ciphertext — useless without the master key. Back up agent.db freely.

MASTER KEY

OS keychain (preferred)

macOS Keychain / Linux Secret Service / Windows Credential Manager. Platform-managed. If you opt out, it's a chmod-600 file at agent-data/master.key.

IN USE

Decrypted only to sign

The agent decrypts in-memory, signs the transaction, wipes the plaintext immediately. Never written back. Never exposed to chat or logs.

The backup gate — three steps.

STEP 01 · REVEAL

./scripts/reveal-secret.sh

Runs in your terminal only. Prints the key once. Doesn't log it. Doesn't echo to any file. Copy it to a password manager, paper, or hardware wallet.

Live trading: STILL REFUSED.

STEP 02 · SAVE

Somewhere durable

1Password. Bitwarden. A printed card in a safe. A hardware wallet. Wherever you already keep the keys you trust.

If your laptop dies, this is what lets you recover.

STEP 03 · CONFIRM

./scripts/confirm-backup.sh

Flips backup_confirmed_at in the database. You're asserting 'yes, I saved it, I can recover.'

Live trading: UNLOCKED for this wallet.

```
# 1. Confirm the wallet exists and is backup-confirmed
> list my wallets

● Using tool: list_wallets
  wallet_id: w-abc123
  address: 0x4f3c...8e21
  backup_confirmed_at: 2026-04-24 10:42:18Z   ✓
  live_enabled: true                           ✓

# 2. Fund it (from exchange, another wallet, or bridge)
#    Send ~$5 USDC to the address above on Base mainnet.

# 3. Verify funds landed on-chain
> check my portfolio

● Using tool: get_portfolio
  USDC: 5.00    ($5.00)
  ETH:  0.0008  (~$2.00, for gas)
```

Two checkmarks above + funds in get_portfolio = you're clear to go live. Either one missing = stop.

CHAPTER 07

Promote a paper strategy to live. Watch one real swap. Verify on-chain. 20 minutes.

Four gates to pass.

GATE 01

The strategy paper-traded cleanly

At least one full cycle of its timeframe. Entries AND exits both fired. Paper trades in list_trades look sensible. If you see nothing but errors, do NOT promote.

GATE 02

The wallet is backup-confirmed

Chapter 6, all three steps. If backup_confirmed_at is NULL, the live-promotion call will refuse with a clear error. That refusal is the gate working.

GATE 03

You've set a small allocation

First live allocation: $1–5 USDC. You're not trying to make money yet. You're verifying the mechanics work on real funds. Scale up after you've seen 2–3 clean fills.

Paper → Live — one promotion call.

FROM PAPER

Scheduler is running. Evaluations firing on the timeframe. Fills are simulated at market price. Zero on-chain activity. Zero funds at risk.

This is where you've been. It's safe. Staying here is fine.

TO LIVE

Ask the bot: "Promote <strategy_id> to live, allocate $5 USDC from <wallet_id>, confirm=true."

Four requirements: user asked · wallet backup-confirmed · allocation complete · confirm=true explicit. All four or it refuses.

Scheduler keeps firing. Fills are now real swaps on Base.

The on-chain flow.

STEP 01

Quote + approve

Agent asks mangrovemarkets for a quote. If it's a new token pair, signs an ERC-20 approval so the router can move funds. Waits for approval confirmation.

STEP 02

Prepare + sign

Agent asks for the unsigned swap payload with your slippage cap. Signs locally with the decrypted key (in-memory only). The signed tx exists for one function call.

STEP 03

Broadcast + log

Signed tx goes to mangrovemarkets for broadcast. Agent polls for confirmation. On success, writes a row to trades with mode=live and the tx_hash. You can open it on basescan.

```
# 1. Promote
> Promote strategy s-ab12 to live. Allocate 5 USDC from w-abc123. confirm=true

● Using tool: update_strategy_status
  status: live, allocation: 5 USDC, wallet: w-abc123
  ✓ Promoted. Next eval at top of hour.

# 2. Force a tick (don't wait)
> Run evaluate_strategy on s-ab12.

● Using tool: evaluate_strategy
  Orders produced: 1 (entry)
● Using tool: execute_swap
  side: buy, input: 5 USDC, output: 0.00198 ETH
  tx_hash: 0x7a9c...4f01
  ✓ Confirmed in block 8,224,551

# 3. Verify on basescan
> Open basescan.org/tx/0x7a9c...4f01 in a browser.
```

If all three steps return clean, you've made a live trade. Watch it for a full cycle before you scale the allocation up.

```
# Through the bot — most common
> Show me evaluations for strategy s-ab12.
● Using tool: list_evaluations
  ev-001  ok   0 orders  287ms   (no entry signal)
  ev-002  ok   1 order   401ms   rsi_cross_up fired
  ev-003  ok   1 order   442ms   sma_cross_down (exit)

> Show me live trades only.
● Using tool: list_trades
  tr-22  live  buy   5 USDC  → 0.00198 ETH  0x7a9c...4f01
  tr-23  live  sell  0.00198 ETH → 5.12 USDC 0xb321...9e08

# Directly, if you want to run SQL
$ sqlite3 agent-data/agent.db
sqlite> SELECT * FROM evaluations WHERE strategy_id='s-ab12';
sqlite> SELECT mode, side, tx_hash FROM trades ORDER BY ts DESC LIMIT 5;
```

The bot's summary = its interpretation. The SQLite = the truth. When they disagree, query directly.

Make it yours.

Add a custom signal: drop a Python file into server/src/signals/, register it, use it in a strategy. The bot wraps Mangrove's signal library — it doesn't restrict you from adding your own.

Add a new MCP tool: the MCP server in server/src/mcp/tools.py is just Python. Add a function, decorate it, restart the server, Claude Code picks it up on next session. You can give the bot read access to news feeds, sentiment data, on-chain metrics — anything you can hit from code.

Extend the safety net: write a hook. A PreToolUse hook on execute_swap that refuses trades above a dollar threshold. A SessionStart hook that pauses all strategies if the bot's been offline more than an hour. Your rules, your guardrails.

This is the end of the tutorial — and the start of the curriculum. Whatever you build next, tell me about it. The Bots & Bytes series gets shaped by what people actually try.

ACT 05

SAFETY IN PRACTICE.

Not for this bot — for every agent you'll build after today. The transferable mental model.

Four principles for agents that touch real things.

01

Self-custody

You hold your own keys, wallets, tokens, credentials. If a hosted service is breached, the worst case is you can't use their API — not 'your money is gone.'

02

Full audit trail

Every evaluation, every tool call, every decision in a database you can query. If the agent claims it did something, the row is there. If it isn't, it didn't happen.

03

Guardrails you can't skip

Hooks. Validators. Permission prompts. Structural refusals, not 'please don't.' The key-paste block refuses your message — it doesn't ask you to be careful.

04

Staged promotion

Draft → backtest → paper → live. Each stage has entry criteria. Never skip stages for speed. Every incident you've heard about skipped a stage.

TAKE THIS HOME

Resources.

THE REPO

github.com/MangroveTechnologies/app-in-a-box

The whole thing you just built. README + tutorials/trading-app/.

MANGROVE DEVELOPER

mangrovedeveloper.ai

Docs, API keys, signal KB, reference strategies.

CLAUDE CODE DOCS

docs.claude.com/claude-code

Skills, hooks, MCP, plan mode — official reference.

CURRICULUM FEEDBACK

tim.darrah@mangrove.ai

Tell me what worked, what didn't, what's missing.

Mangrove®

THAT'S THE WORKSHOP

Thank you.

Questions, feedback, war stories — right now.

Tim Darrah · tim.darrah@mangrove.ai · mangrovedeveloper.ai